Binaire et hexadécimal

Comment les nombres sont-ils représentés dans un ordinateur ?

$$(1100'1010'1111'1110)_2 = (51'966)_{10} = (CAFE)_{16}$$

Nicholas Wolff · CC BY-SA 4.0

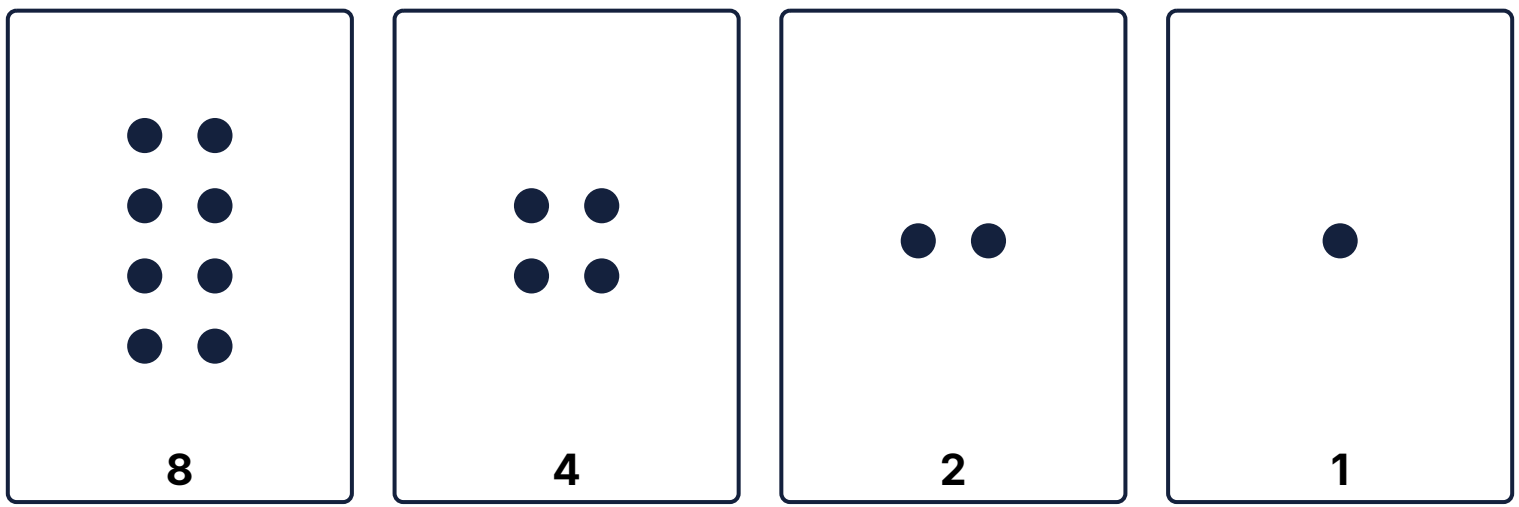
Représentation des nombres dans un ordinateur

Des bits aux nombres

1. **Compter**
   Bases et notations
2. **Convertir**
   Binaire ↔ décimal
3. **Additionner**
   En colonnes
4. **Multiplier et diviser**
   Décaler les bits
5. **Nombres négatifs**
   Le complément à deux
6. **Hexadécimal**
   Le binaire en plus compact

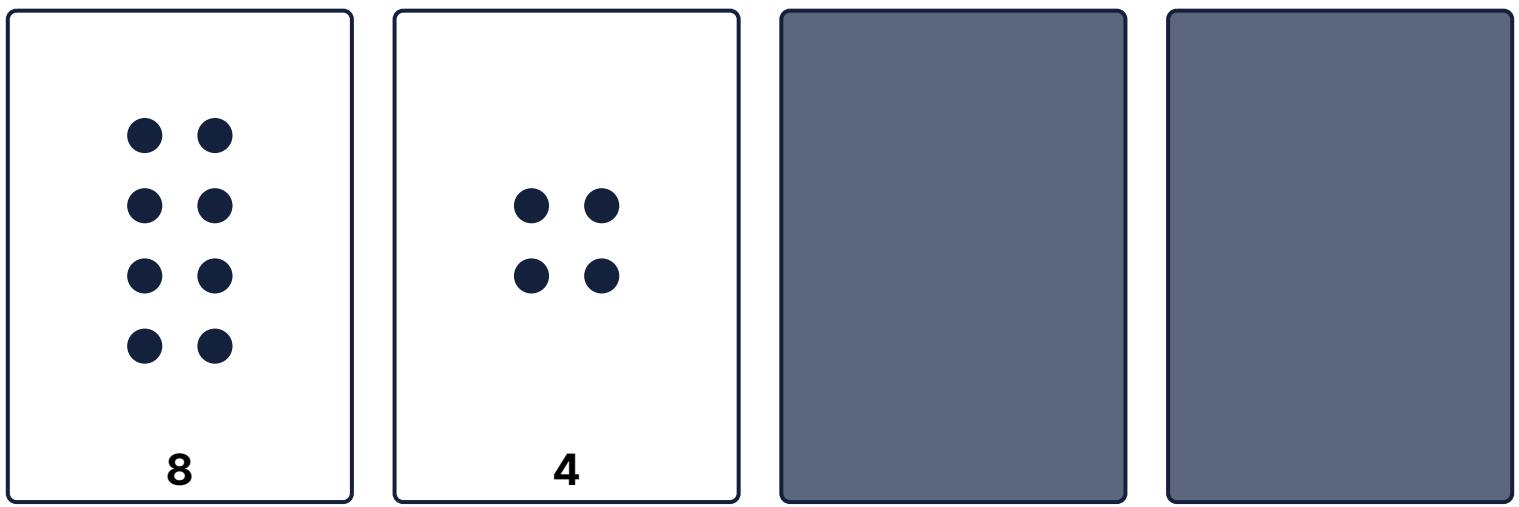
Activité débranchée

Disposer les cartes dans l’ordre, la plus grande à gauche.



Activité débranchée

On code les cartes avec des 0 (carte de dos) et des 1 (les points sont visibles)



1100 → 8 + 4 = 12

Activité débranchée

Activité · 8 min

Au tableau et de vive voix

Les symboles pour compter

Activité · 1 min

En quelle base compte-t-on habituellement ? Avec quels symboles ?

Les symboles pour compter

En quelle base compte-t-on habituellement ? Avec quels symboles ?

Décimal · base 10

Un nombre entier décimal s'écrit uniquement avec les dix symboles

`0 1 2 3 4 5 6 7 8 9`

Exemple : `124'509`

Binaire · base 2

Un nombre entier binaire s'écrit uniquement avec les deux symboles

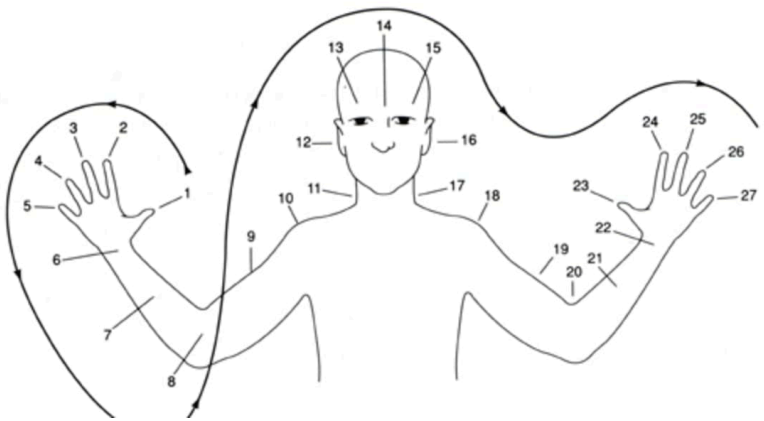
`0 1`

Exemple : `1'0011`

D’autres bases

On peut compter dans n’importe quelle base, et selon les cultures, on l’a fait :

- **base 8** — Les Yuki, en Californie : les espaces entre les doigts
- **base 20** — Les Mayas ; en français : quatre-vingts
- **base 60** — Les Babyloniens, en Mésopotamie
- **base 27** — Les Oksapmin, en Papouasie-Nouvelle-Guinée : sur le corps



Compter jusqu’à 27 sur le corps (Oksapmin)

Activité · 4 min

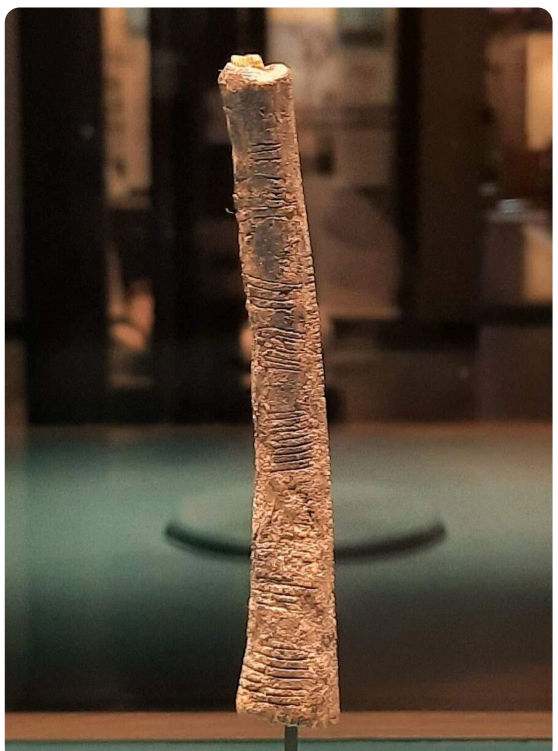
Quelles bases autres que 10 utilisons-nous dans la vie de tous les jours ?

La base 10 n’est qu’un choix, sans doute dicté par nos dix doigts. Toutes les bases permettent d’écrire les mêmes nombres.

Compter sans base

On ajoute une coche pour chaque nouvel objet compté. Pour savoir où on en est, il faut tout recompter depuis le début.

L’os d’Ishango, en République démocratique du Congo : il y a plus de 20'000 ans, l’un des plus anciens objets de comptage connus. Il est exposé à Bruxelles.



Une autre notation : les chiffres romains

C’est à peu près la base 10, mais avec une autre notation.

I	V	X	L	C	D	M
1	5	10	50	100	500	1'000

On additionne les symboles

LXVII = 50 + 10 + 5 + 1 + 1 = **67**

Un symbole plus petit placé avant un plus grand se soustrait

CDXII = (500 − 100) + 10 + 1 + 1 = **412**

Un symbole ne se répète pas plus de trois fois

40 = **XL**, et non **XXXX**

Activité · 3 min

Calculer **LXXXVII + XI**, en chiffres romains. Comment avez-vous procédé ?

La notation positionnelle

La valeur d'un chiffre dépend de sa position : son **poids**, une puissance de la base.

En décimal · les poids sont des puissances de 10

poids	10^3 1'000	10^2 100	10^1 10	10^0 1
chiffre	**7**	**2**	**4**	**8**

$(7'248)_{10} = 7 \times 1'000 + 2 \times 100 + 4 \times 10 + 8 \times 1$

En binaire · les poids sont des puissances de 2

poids	2^5 32	2^4 16	2^3 8	2^2 4	2^1 2	2^0 1
chiffre	**1**	**0**	**0**	**1**	**1**	**0**

$(100110)_2 = 32 + 4 + 2 = (38)_{10}$

Définitions : bit et octet

Le bit

La plus petite unité d'information : `0` ou `1`.

Contraction de l'anglais *binary digit*, chiffre binaire.

L'octet · *byte* en anglais

Huit bits.

Les nombres de ce cours tiennent souvent sur un octet.

Exemples

- `1001'0001` est un octet ; il vaut `145` en décimal.
- `111` aussi. On peut le compléter à gauche avec des `0` : `0000'0111`.
- `1'0010'1100` n'est pas un octet : il a 9 bits.

On groupe les bits par 4 avec une apostrophe, pour les lire plus facilement.

Sommaire

1 **Compter**
Bases et notations

2 **Convertir**
Binaire ↔ décimal

3 **Additionner**
En colonnes

4 **Multiplier et diviser**
Décaler les bits

5 **Nombres négatifs**
Le complément à deux

6 **Hexadécimal**
Le binaire en plus compact

La grille de conversion

Une grille pour convertir dans les deux sens : chaque colonne a un poids.

Binaire → décimal, par exemple `1'0011` :

1. Écrire chaque bit sous son poids, en partant de la droite.
2. Dans « Calculs », noter les poids des bits à 1.
3. Les additionner : c'est la valeur en décimal.

8 bits, soit un octet

128	64	32	16	8	4	2	1	Décimal	Calculs
			1	0	0	1	1	19	16 + 2 + 1

Le plus grand nombre sur un octet

Activité · 3 min

Quel est le plus grand nombre qu'on peut écrire avec 8 bits ?

Le plus grand nombre sur un octet

Quel est le plus grand nombre qu'on peut écrire avec 8 bits ?

Il y a trois façons de trouver le résultat :

Tous les bits à 1

$(1111'1111)_2$ = 128 + 64 + 32 + 16 + 8 + 4 + 2 + 1 = $(255)_{10}$

Le nombre suivant

$(1'0000'0000)_2$ = $(256)_{10}$: un 9e bit, de poids 256. Donc 256 − 1 = **255**.

Les combinaisons

Chaque bit a 2 valeurs : 2 × 2 × … × 2 = 2^8 = 256 combinaisons, pour les nombres de 0 à **255**.

Avec *n* bits, on écrit 2^n nombres différents, de 0 à $2^n - 1$.

Décimal → binaire, par soustractions successives

Convertir **76** en binaire, en partant des poids les plus forts :

Poids			Bit	Reste à convertir
128	rentre dans 76 ?	non	0	
64	rentre dans 76 ?	oui	1	76 − 64 = **12**
32	rentre dans 12 ?	non	0	
16	rentre dans 12 ?	non	0	
8	rentre dans 12 ?	oui	1	12 − 8 = **4**
4	rentre dans 4 ?	oui	1	4 − 4 = **0**
2	rentre dans 0 ?	non	0	
1	rentre dans 0 ?	non	0	

8 bits, soit un octet

128	64	32	16	8	4	2	1
0	1	0	0	1	1	0	0

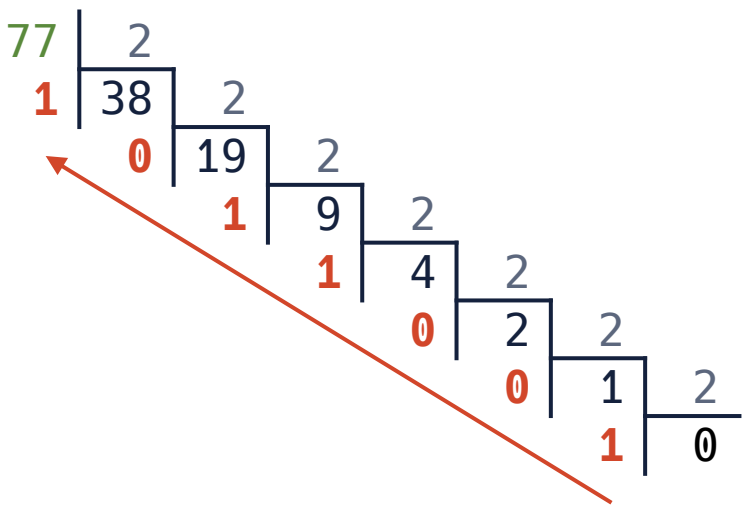
Résultat : **76** s'écrit **`100'1100`** en binaire.

Le 0 de gauche ne change rien à la valeur : on peut l'omettre.

Décimal → binaire, par divisions successives par 2

Convertir **77** en binaire :

1. Diviser par 2, puis diviser le quotient par 2, et ainsi de suite, jusqu'à ce que le **quotient** soit 0.
2. Les **restes** sont les bits : les collecter du bas vers le haut.



Résultat : **77** s'écrit **100'1101** en binaire.

Exercices : binaire ↔ décimal

Activité · 8 min

Avec la grille de conversion :

1. Convertir `100'0001` en décimal
2. Convertir `1100'1101` en décimal
3. Convertir `131` en binaire
4. Convertir `151` en binaire
5. Convertir `255` en binaire
6. Convertir `0` en binaire
7. Convertir `256` en binaire

Exercices : binaire ↔ décimal

Les réponses, dans l'ordre des exercices :

8 bits, soit un octet									
128	64	32	16	8	4	2	1	Décimal	Calculs
	1	0	0	0	0	0	1	65	64 + 1
1	1	0	0	1	1	0	1	205	128 + 64 + 8 + 4 + 1
1	0	0	0	0	0	1	1	131	128 + 2 + 1
1	0	0	1	0	1	1	1	151	128 + 16 + 4 + 2 + 1
1	1	1	1	1	1	1	1	255	128 + 64 + 32 + 16 + 8 + 4 + 2 + 1
							0	0	0

256 ne tient pas sur un octet : il faut un 9ᵉ bit, de poids 256. Il s'écrit **1'0000'0000**.

Sommaire

1 **Compter** Bases et notations

2 **Convertir** Binaire ↔ décimal

3 **Additionner** En colonnes

4 **Multiplier et diviser** Décaler les bits

5 **Nombres négatifs** Le complément à deux

6 **Hexadécimal** Le binaire en plus compact

L’addition en colonnes

Activité · 3 min

Au tableau : calculer `365 + 1'478`, en expliquant chaque étape.

L’addition en colonnes

Rappel, en décimal

1. Aligner les deux nombres à droite.
2. Additionner colonne par colonne, en partant de la droite, avec parfois une retenue.

```
  1 1
 1478
+ 365
------
 1843
```

En binaire, la même méthode

Il suffit de connaître les sommes de deux bits, ou de trois avec une retenue :

```
0 + 0 =  0
0 + 1 =  1
1 + 0 =  1
1 + 1 = 10
1 + 1 + 1 = 11
```

1 + **1** = **10** : on écrit 0, et on retient 1.

L'addition binaire

1. Écrire les nombres en colonnes, alignés à droite, le plus grand en haut.
2. Additionner colonne par colonne, en partant de la droite.
3. Si la somme d'une colonne dépasse 1, elle s'écrit sur deux bits (**10** ou **11**) : écrire le bit de droite, retenir 1 dans la colonne suivante.
4. Continuer jusqu'à la dernière colonne.
5. Vérifier en passant par le décimal : convertir les nombres, puis refaire le calcul en décimal.

Exemple

```
  1 1 1 1
  11011011
+     1110
----------
  11101001
```

Vérification

a.	Convertir les deux termes et la somme en décimal	$(1101'1011)_2 = (219)_{10}$ $(1110)_2 = (14)_{10}$ $(1110'1001)_2 = (233)_{10}$
b.	Calculer en décimal	$219 + 14 = 233$
c.	Vérifier que les deux résultats sont égaux	$233 = 233$ ✓

Exercices : addition binaire

Activité · 7 min

Les nombres sont représentés sur 8 bits.

1. Calculer en binaire `1'1101` + `110`
2. Vérifier en passant par le décimal : convertir les nombres, puis refaire le calcul en décimal.
3. Calculer en binaire `1'0111` + `100'0111`
4. Calculer en binaire `1` + `1111'1111`

Les sommes de bits

```
    0 + 0 =  0
    0 + 1 =  1
    1 + 0 =  1
    1 + 1 = 10
1 + 1 + 1 = 11
```

Exercices : addition binaire

Exercice 1

```
  1 1 1
   11101
+    110
------
  100011
```

Exercice 2

$(1'1101)_2 = (29)_{10}$

$(110)_2 = (6)_{10}$

$(10'0011)_2 = (35)_{10}$

```
  1
  29
+  6
----
  35
```

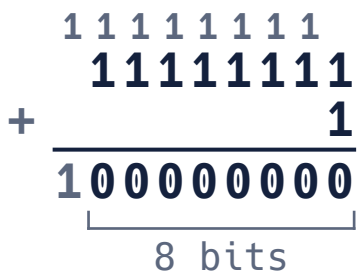
$35 = 35$: la somme est juste.

Exercice 3

```
      1 1 1
  1000111
+   10111
-------
  1011110
```

Le dépassement de capacité

Les nombres sont représentés sur **8 bits**.



Vérification en décimal

```
  255
+   1
  ---
    0 ?!
```

Le bit qui dépasse ne tient pas sur 8 bits : il est perdu. On parle de **dépassement de capacité**.

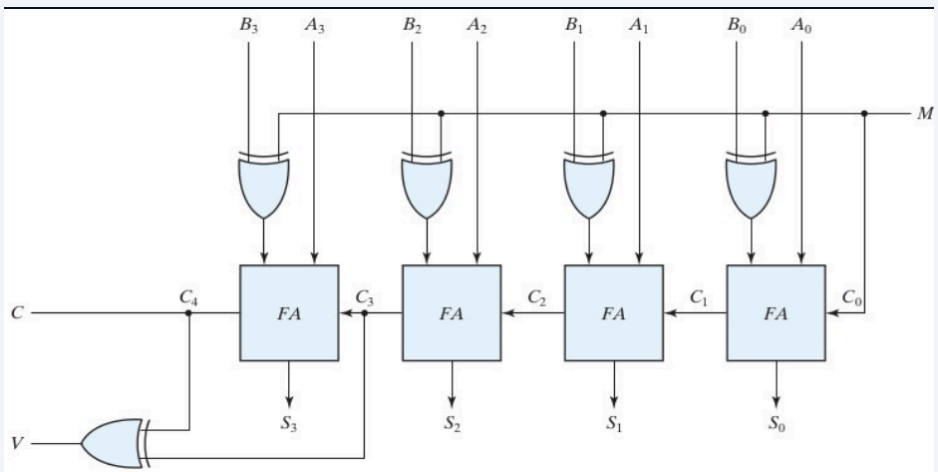
Le dépassement de capacité, dans des machines

Un compteur mécanique



Après 99'999 : 00'000.

Un processeur



Un additionneur de 4 bits : la dernière retenue (C) n'entre pas dans le résultat (S_3 à S_0).

Sommaire

1 **Compter** — Bases et notations

2 **Convertir** — Binaire ↔ décimal

3 **Additionner** — En colonnes

4 **Multiplier et diviser** — Décaler les bits

5 **Nombres négatifs** — Le complément à deux

6 **Hexadécimal** — Le binaire en plus compact

La multiplication la plus simple

Activité · 1 min

En décimal, quelle est la multiplication la plus simple, à part par 0 et par 1 ?

La multiplication la plus simple

7'248	
72'480	× 10
724'800	× 100
7'248'000	× 1'000

Multiplier par 10, 100, 1'000…, c’est ajouter des zéros à droite : chaque chiffre se décale vers la gauche, et son poids est multiplié par 10 à chaque colonne.

Activité · 1 min

Et en binaire : ajouter un zéro à droite, c’est multiplier par combien ?

Multiplier par des puissances de deux

Par exemple :

× 2 : on ajoute un zéro à droite. Tous les bits se décalent d'une colonne vers la gauche.

× 4 = × 2 × 2 : on ajoute deux zéros à droite. Les bits se décalent de deux colonnes.

2^8	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
256	128	64	32	16	8	4	2	1
		1	0	0	0	1	1	0
	1	0	0	0	1	1	0	0
1	0	0	0	1	1	0	0	0

Décimal	Calculs
70	64 + 4 + 2
140	128 + 8 + 4
280	256 + 16 + 8

Chaque bit à 1 double son poids : la somme double aussi.



Multiplier par 2^n, c'est ajouter n zéros à droite. Tous les bits se décalent de n colonnes vers la gauche.

Multiplier par n'importe quel nombre

On veut calculer 1011×101.

Le multiplicateur se décompose en puissances de 2 : $101 = 100 + 1$ (5 = 4 + 1), donc :

$$1011 \times 101 = 1011 \times 100 + 1011 \times 1$$

$1011 \times 100 = 101100$ deux zéros à droite : décalé de deux colonnes

$1011 \times 1 = 1011$ le nombre lui-même, sans décalage

Chaque 1 du multiplicateur donne une copie du nombre, décalée selon sa position. On additionne les copies.

Activité · 2 min

Additionner les deux lignes, puis vérifier la multiplication binaire en passant par le décimal.

La multiplication binaire

On veut calculer **111011** × **110** :

```
    111011
×       110
-----------
         0
+   111011·
+  111011··
-----------
 101100010
```

a) **111011** × 0 : la ligne vaut 0.

b) Un point, qui remplace un 0, c) puis **111011** × 1 : on recopie le nombre.

d) Deux points, e) puis **111011** × 1 : on recopie encore.

f) On additionne les lignes.

On peut vérifier : **11'1011** × **110** = **1'0110'0010**

En passant par le décimal : **59** × **6** = **354**

Exercices : multiplication binaire

Activité · 8 min

1. Calculer en binaire `1'0101` × `101`
2. Vérifier en passant par le décimal : convertir les nombres, puis refaire le calcul en décimal.
3. Calculer en binaire `10` × `10'1010'1101'1101`

   Réfléchir avant de se lancer dans un long calcul : il y a une façon rapide de faire.

Exercices : multiplication binaire

1. Calculer en binaire `1'0101` × `101`

```
    10101
×     101
-------
    10101
+      0··
+ 10101··
-------
 1101001
```

2. Vérifier en passant par le décimal.

Binaire	Décimal
1'0101	21
101	5
110'1001	105

`21 × 5 = 105` : le calcul binaire est juste.

Exercices : multiplication binaire

3. Calculer en binaire `10` × `10'1010'1101'1101`

Le long nombre en haut : deux lignes seulement.

```
  101010110111 01
×              10
----------------
               0
+ 101010110111 01·
----------------
  101010110111 010
```

Plus rapide : `10` vaut `2`, et multiplier par 2, c'est ajouter un zéro à droite.

`10'1010'1101'1101` × `10`
= `101'0101'1011'1010`

Diviser par des puissances de deux

Par exemple :

÷ 2 : on décale les bits d'une colonne vers la droite. Le bit de droite disparaît.

÷ 4 = ÷ 2 ÷ 2 : on décale les bits de deux colonnes. Les deux bits de droite disparaissent.

2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0	Décimal	Calculs
128	64	32	16	8	4	2	1		
1	0	0	0	1	1	0	0	140	128 + 8 + 4
	1	0	0	0	1	1	0	70	64 + 4 + 2
		1	0	0	0	1	1	35	32 + 2 + 1



Diviser par 2^n (division entière), c'est enlever les n bits de droite. Tous les bits se décalent de n colonnes vers la droite.

Exercices : division binaire

Activité · 8 min

1. Diviser `1'0100` par 2. Vérifier en passant par le décimal : convertir les nombres, puis refaire le calcul en décimal.
2. Diviser `1'0101` par 2. Vérifier en passant par le décimal : convertir les nombres, puis refaire le calcul en décimal.
   Y a-t-il une erreur ? Qu'en pensez-vous ?
3. Diviser `1'0000` par 16.
4. Diviser `1111` par 16.

Exercices : division binaire

1. Diviser `1'0100` par 2. Vérifier en passant par le décimal.

128	64	32	16	8	4	2	1	Décimal	Calculs
			1	0	1	0	0	20	16 + 4
				1	0	1	0	10	8 + 2

2. Diviser `1'0101` par 2. Vérifier en passant par le décimal. Y a-t-il une erreur ?

128	64	32	16	8	4	2	1	Décimal	Calculs
			1	0	1	0	1	21	16 + 4 + 1
				1	0	1	0	10	8 + 2

Pas d'erreur : la division est entière. Le bit qui disparaît, 1, est le reste.

Exercices : division binaire

3. Diviser `1'0000` par 16.

128	64	32	16	8	4	2	1	Décimal	Calculs
			1	0	0	0	0	16	16
							1	1	1

Les 4 bits qui disparaissent, `0000`, sont le reste : 16 ÷ 16 = 1, reste 0.

4. Diviser `1111` par 16.

128	64	32	16	8	4	2	1	Décimal	Calculs
				1	1	1	1	15	8 + 4 + 2 + 1
							0	0	0

Tous les bits disparaissent : 15 ÷ 16 = 0, reste 15.

Pourquoi la conversion par divisions fonctionne-t-elle ?

On veut convertir $(77)_{10}$ en binaire :

	Décimal	64	32	16	8	4	2	1	Reste
÷ 2	77	1	0	0	1	1	0	1	
÷ 2	38		1	0	0	1	1	0	1
÷ 2	19			1	0	0	1	1	0
÷ 2	9				1	0	0	1	1
÷ 2	4					1	0	0	1
÷ 2	2						1	0	0
÷ 2	1							1	0
	0								1

$$(77)_{10} = (100'1101)_2$$

Diviser par 2 décale les bits d'une colonne vers la droite, et le bit qui disparaît est le reste. Les restes sont donc les bits du nombre, de droite à gauche : on les collecte du bas vers le haut.

Sommaire

1 **Compter** Bases et notations

2 **Convertir** Binaire ↔ décimal

3 **Additionner** En colonnes

4 **Multiplier et diviser** Décaler les bits

5 **Nombres négatifs** Le complément à deux

6 **Hexadécimal** Le binaire en plus compact

Qu’est-ce qu’un bit d’information ?

Au jeu « Qui est-ce ? », on pose des questions dont la réponse est **oui** ou **non**.

Chaque réponse donne **1 bit d’information**.

En combinant plusieurs réponses, on trouve le personnage.



Un personnage en 4 bits

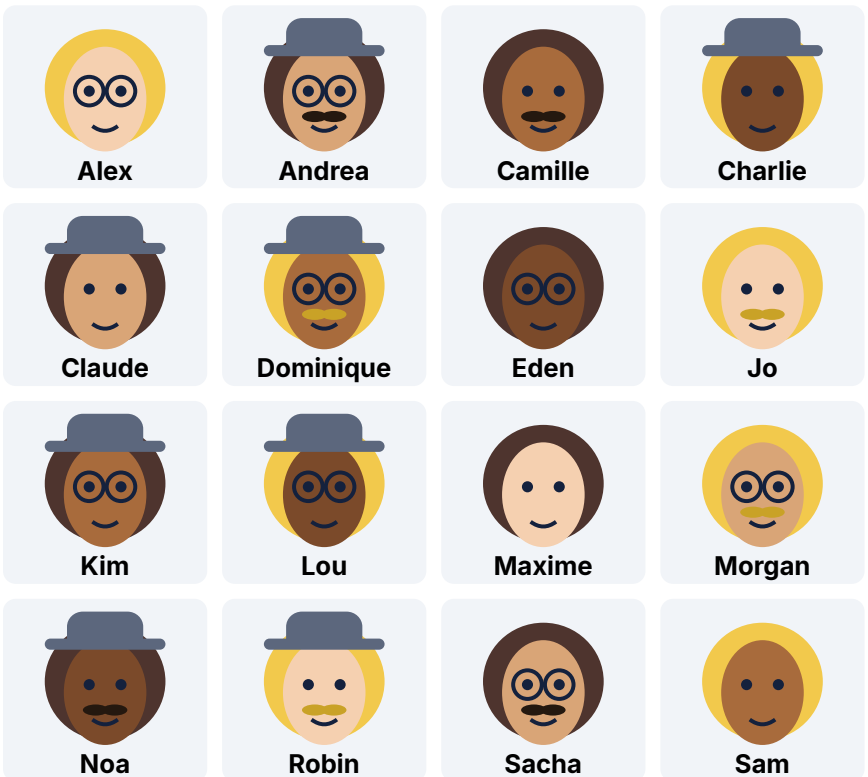
On pose toujours les mêmes questions, dans le même ordre. Chaque réponse est un bit : **1** pour oui, **0** pour non.

1. Cheveux blonds ?
2. Chapeau ?
3. Lunettes ?
4. Moustache ?

Alex s'écrit `1010` : oui, non, oui, non.

Activité · 3 min

1. Qui est `0110` ?
2. Quel est le code de Lou ?
3. Combien de personnages peut-on distinguer avec 4 questions ?



Et les nombres négatifs ?

Pour commencer, et pour simplifier : on n'a que **4 bits** à disposition.

On sait écrire des entiers positifs en binaire :

$$(0000)_2 = (0)_{10} \qquad (0101)_2 = (5)_{10} \qquad (1111)_2 = (15)_{10}$$

On aimerait aussi écrire des nombres négatifs : −1, −3… Mais un ordinateur n'a que des 0 et des 1, pas de signe −.

À discuter · 3 min

Quelle idée pourrait-on avoir ?

Première idée : signe et valeur absolue

Le bit de gauche donne le **signe** : il est à 1 pour un nombre négatif.
Les 3 bits suivants donnent la **valeur absolue**, le nombre sans son signe (celle de −3 est 3).

Signe	4	2	1	Décimal
0	0	1	1	3
1	0	1	1	−3
				−4
				5
				0

Activité · 5 min

1. Compléter le tableau.
2. Quel est le plus grand nombre qu'on peut représenter ?
3. Quel est le plus petit nombre qu'on peut représenter ?
4. Combien de nombres différents peut-on représenter au total ?

Première idée : signe et valeur absolue

Le bit de gauche donne le **signe** : il est à 1 pour un nombre négatif.
Les 3 bits suivants donnent la **valeur absolue**, le nombre sans son signe (celle de −3 est 3).

Signe	4	2	1	Décimal
0	0	1	1	3
1	0	1	1	−3
1	1	0	0	−4
0	1	0	1	5
0	1	1	1	7
1	1	1	1	−7
0	0	0	0	0
1	0	0	0	0

2. Quel est le plus grand nombre qu'on peut représenter ? `7`, qui s'écrit `0111`.
3. Quel est le plus petit nombre qu'on peut représenter ? `−7`, qui s'écrit `1111`.
4. Combien de nombres différents peut-on représenter au total ? **15** : 4 bits donnent 16 combinaisons, mais deux d'entre elles, `0000` et `1000`, représentent le même nombre, zéro.

Signe et valeur absolue : additionner

Avec ce système qu'on a inventé, essayons d'additionner deux nombres.

Signe	4	2	1	Décimal
0	0	1	1	3
1	0	1	1	−3

Activité · 2 min

Calculer `3 + (−3)` en binaire, sur 4 bits, puis convertir le résultat en décimal.

Signe et valeur absolue : deux problèmes

Deux zéros

+0 s'écrit 0000 −0 s'écrit 1000

Deux combinaisons pour le même nombre : une est gaspillée.

L'addition ne marche plus

```
  1 1
  0011
+ 1011
------
  1110
```

3 + (−3) devrait donner 0.
On obtient 1110, soit −6 !

Cette représentation n'est vraiment pas utile : **ce n'est pas celle que les ordinateurs utilisent**.

Le complément à deux

La représentation utilisée par tous les ordinateurs. Le bit de gauche donne toujours le signe : les positifs s'écrivent comme avant, avec un bit de signe à 0. Pour un **négatif** :

Exemple : −3

a.	Prendre la valeur absolue	**3**
b.	Soustraire 1	**2**
c.	Écrire ce résultat en binaire sur 4 bits, avec des 0 à gauche si besoin	**0010**
d.	Inverser tous les bits : les 0 deviennent des 1, et inversement	**1101**

Vérification : le bit de signe de **1101** est bien à 1, c'est un nombre négatif.

Activité · 4 min

Écrire sur 4 bits : **−1** et **−5**.

Le complément à deux

Écrire sur 4 bits :



Sur 4 bits, −1 s'écrit 1111



Sur 4 bits, −5 s'écrit 1011

Le complément à deux règle les deux problèmes

Un seul zéro

`0` s'écrit `0000`

`1000` n'est plus « −0 » : il représente un autre nombre. Aucune combinaison n'est gaspillée.

L'addition marche

```
  1 1 1 1
    0 0 1 1
+   1 1 0 1
  ---------
  1 0 0 0 0
    └─────┘
    4 bits
```

3 + (−3) doit donner 0.
On garde 4 bits : `0000` vaut `0`.

Du binaire au décimal

D’abord, regarder le bit de signe : à 0, le nombre est positif et se convertit comme avant. À 1, il est **négatif** :

Exemple : `1001`

a.	Inverser tous les bits	**`0110`**
b.	Convertir ces bits en décimal	**6**
c.	Ajouter 1	**7**
d.	Mettre un signe − devant	**−7**

Le complément à deux en résumé

Décimal → binaire, pour un nombre négatif



Binaire → décimal, quand le bit de signe est à 1



Cet exemple utilise 4 bits, mais la technique est la même quel que soit le nombre de bits.

Exercices : vers le décimal

Activité · 6 min

Convertir en décimal ces nombres de 4 bits, positifs ou négatifs, en complément à deux :

1. `0110`
2. `1110`
3. `1100`
4. `11`

Rappel, pour un nombre négatif :



Exercices : vers le décimal

Convertir en décimal ces nombres de 4 bits, positifs ou négatifs, en complément à deux :

1. `0110` : bit de signe à 0 : positif. Avec la grille : 4 + 2 = **6**.
2. `1110` : bit de signe à 1 : négatif.



3. `1100` : bit de signe à 1 : négatif.



4. `11` s'écrit `0011` sur 4 bits : bit de signe à 0 : positif. Avec la grille : 2 + 1 = **3**.

Additionner en complément à deux

Exemple : −3 + 5

```
  1 1  1
   1101
+  0101
------
  10010
   └──┘
  4 bits
```

`1101` vaut `−3`
`0101` vaut `5`

On garde 4 bits :
`0010` vaut `2`

Le bit qui dépasse est ignoré : en complément à deux, ce n'est pas une erreur.

Activité · 8 min

Calculer en binaire, sur 4 bits, puis vérifier en décimal :

1. `−4 + (−2)`
2. `−1 + 1`
3. `−7 + 5`

Convertir les deux nombres en binaire, les additionner, garder 4 bits, puis reconvertir le résultat.

Rappel, pour un nombre négatif :

`−6` → valeur absolue → `6` → soustraire 1 → `5` → en binaire, 4 bits → `0101` → inverser les bits → `1010`

`1010` → inverser les bits → `0101` → en décimal → `5` → ajouter 1 → `6` → signe − → `−6`

Additionner en complément à deux

Calculer en binaire, sur 4 bits, puis vérifier en décimal :

−4 + (−2)

```
  1 1
  1100
+ 1110
-------
 11010
 ----
 4 bits
```

1100 vaut −4
1110 vaut −2

On garde 4 bits : 1010 vaut −6

Vérification : −4 + (−2) = −6

−1 + 1

```
  1 1 1 1
  1111
+ 0001
-------
 10000
 ----
 4 bits
```

1111 vaut −1
0001 vaut 1

On garde 4 bits : 0000 vaut 0

Vérification : −1 + 1 = 0

−7 + 5

```
    1
  1001
+ 0101
-------
  1110
  ----
  4 bits
```

1001 vaut −7
0101 vaut 5

On garde 4 bits : 1110 vaut −2

Vérification : −7 + 5 = −2

Une nouvelle opération

On sait additionner. On sait écrire les nombres négatifs, et l'addition marche avec eux.

À discuter · 1 min

Quelle nouvelle opération vient de s'ouvrir à nous ?

Une nouvelle opération : la soustraction

On sait additionner. On sait écrire les nombres négatifs, et l'addition marche avec eux. Quelle nouvelle opération vient de s'ouvrir à nous ?

La soustraction : $x - y = x + (-y)$

```
  1
  0010
+ 1011
------
  1101
  ____
  4 bits
```

Exemple : `2 - 5 = 2 + (-5)`

`0010` + `1011` donne `1101`, qui vaut `-3`.

Les nombres de 4 bits en complément à deux

Activité · 3 min

a) Quel est le plus grand nombre qu'on peut représenter ?

b) Quel est le plus petit nombre qu'on peut représenter ?

Essayer de représenter l'opposé du nombre trouvé précédemment.
Y en a-t-il un encore plus petit ?

c) Combien de nombres différents peut-on représenter au total ?

Les nombres de 4 bits en complément à deux

a) Quel est le plus grand nombre qu'on peut représenter ?

Signe	4	2	1	Décimal
0	1	1	1	7

b) Quel est le plus petit nombre qu'on peut représenter ?

Essayer de représenter l'opposé du nombre trouvé précédemment.
Y en a-t-il un encore plus petit ?

Signe	4	2	1	Décimal
1	0	0	1	−7
1	0	0	0	−8

c) Combien de nombres différents peut-on représenter au total ?

Signe	4	2	1	Décimal
0	0	0	0	0

De −8 à 7 : **16 nombres** différents, avec un seul zéro.

Activité · 2 min

Pourquoi y a-t-il un nombre de plus du côté négatif ?

Les nombres de 8 bits et plus en complément à deux

- 4 bits : de **−8** à **7**
- 8 bits : de **−128** à **127**

Signe	64	32	16	8	4	2	1	Décimal
0	1	1	1	1	1	1	1	127
0	0	0	0	0	0	0	0	0
1	1	1	1	1	1	1	1	−1
1	0	0	0	0	0	0	0	−128

- 16 bits : de **−32'768** à **32'767**
- 32 bits : de **−2'147'483'648** à **2'147'483'647**

À retenir : avec *n* bits, on écrit toujours 2^n nombres différents :

- sans signe, de 0 à $2^n - 1$;
- en complément à deux, de -2^{n-1} à $2^{n-1} - 1$.

Le dépassement de capacité, en complément à deux

Activité · 3 min

Calculer $5 + 4$ en binaire, sur 4 bits. Que se passe-t-il ?

Le dépassement de capacité, en complément à deux

Calculer **5 + 4** en binaire, sur 4 bits. Que se passe-t-il ?

```
  1
  0101
+ 0100
------
  1001
  ----
  4 bits
```

0101 vaut **5**, **0100** vaut **4**
1001 vaut **−7** : deux positifs donnent un négatif !

9 n'est pas entre **−8** et **7** : il ne tient pas sur 4 bits. C'est un **dépassement de capacité**, comme pour les nombres sans signe.

En complément à deux, le dépassement ne se voit pas au bit qui sort, mais au résultat : il sort de l'intervalle des nombres représentables.

Sommaire

1 **Compter** Bases et notations

2 **Convertir** Binaire ↔ décimal

3 **Additionner** En colonnes

4 **Multiplier et diviser** Décaler les bits

5 **Nombres négatifs** Le complément à deux

6 **Hexadécimal** Le binaire en plus compact

À quoi sert l'hexadécimal ?

Décimal	Binaire	Hexadécimal
0	0	0
9	1001	9
15	1111	F
16	1'0000	10
27	1'1011	1B
255	1111'1111	FF
256	1'0000'0000	100
200'000	11'0000'1101'0100'0000	30D40

L'hexadécimal écrit les nombres binaires de façon **compacte**.
Deux symboles hexadécimaux suffisent pour un octet : $(1111'1111)_2$ = $(FF)_{16}$.

Compter en base 16

- Compter en hexadécimal, c'est compter en **base 16**, avec 16 symboles :

`0 1 2 3 4 5 6 7 8 9 A B C D E F`

- Un symbole représente les nombres de 0 à 15, soit **4 bits**.
- On combine les symboles pour écrire de plus grands nombres : `4E`, `F0AF`, `109A22`.

Préciser la base

Nous avons maintenant trois façons d'écrire les nombres : binaire (base 2), décimal (base 10), hexadécimal (base 16).

Pour préciser la base, on écrit le nombre entre parenthèses et la base en bas à droite :

Binaire
$(111)_2$ $(101'1010)_2$

Décimal
$(111)_{10}$ $(89)_{10}$

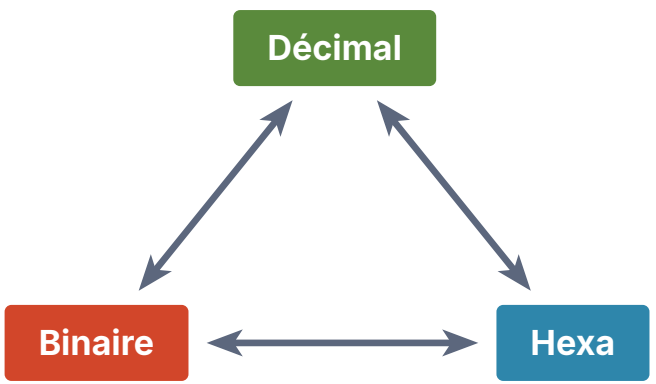
Hexadécimal
$(111)_{16}$ $(89)_{16}$ $(90F)_{16}$ $(CAFEBAD)_{16}$

$(111)_2 = (7)_{10}$ $\quad$ $(111)_{16} = (273)_{10}$

Les 16 symboles

Binaire	Hexa	Décimal
0000	0	0
0001	1	1
0010	2	2
0011	3	3
0100	4	4
0101	5	5
0110	6	6
0111	7	7
1000	8	8
1001	9	9
1010	A	10
1011	B	11
1100	C	12
1101	D	13
1110	E	14
1111	F	15

Les 16 symboles hexadécimaux correspondent aux 16 nombres binaires de 4 bits, et aux nombres décimaux de 0 à 15.



Ce tableau permet de convertir immédiatement les petits nombres entre les trois bases.

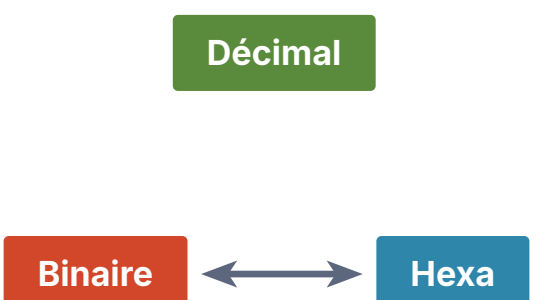
Binaire ↔ hexadécimal

Binaire	Hexa
0000	0
0001	1
0010	2
0011	3
0100	4
0101	5
0110	6
0111	7
1000	8
1001	9
1010	A
1011	B
1100	C
1101	D
1110	E
1111	F

Au-delà de **F**, la conversion reste simple : chaque groupe de 4 bits donne un symbole.

1. Couper le binaire en groupes de 4 bits, **en partant de la droite**.
2. Remplacer chaque groupe par son symbole, avec le tableau.

Dans l'autre direction : remplacer chaque symbole par ses 4 bits, avec le tableau.




```
1111 0011
   F    3
(1111'0011)₂ = (F3)₁₆

10 0101 1110
 2    5    E
(10'0101'1110)₂ = (25E)₁₆
```

Dans l'autre direction

```
   B    2
1011 0010
(B2)₁₆ = (1011'0010)₂
```

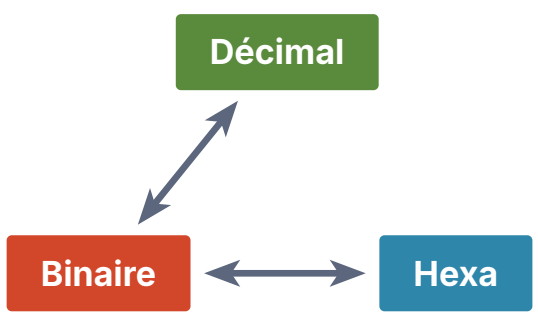
Décimal ↔ hexadécimal, en passant par le binaire

Binaire	Hexa
0000	0
0001	1
0010	2
0011	3
0100	4
0101	5
0110	6
0111	7
1000	8
1001	9
1010	A
1011	B
1100	C
1101	D
1110	E
1111	F

On sait convertir entre décimal et binaire, et entre binaire et hexadécimal.

En enchaînant les deux, on convertit en passant par le binaire :

- de décimal à hexadécimal,
- d'hexadécimal à décimal.




```
      44
  10  1100
   2     C
(44)₁₀ = (2C)₁₆

       873
 11  0110  1001
  3     6     9
(873)₁₀ = (369)₁₆
```

Dans l'autre direction

```
   A     7
1010  0111
       167
(A7)₁₆ = (167)₁₀
```

Hexadécimal → décimal, directement

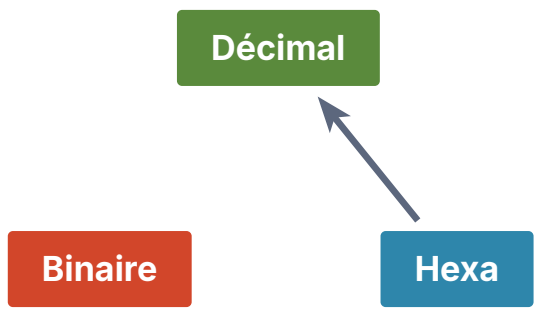
Hexa	Décimal
0	0
1	1
2	2
3	3
4	4
5	5
6	6
7	7
8	8
9	9
A	10
B	11
C	12
D	13
E	14
F	15

On utilise une grille dont les poids sont des puissances de 16 : 1, 16, 256, 4'096, 65'536…

1. Écrire le nombre dans la grille, en partant de la droite.
2. Convertir chaque symbole en sa valeur décimale, avec le tableau.
3. Multiplier chaque valeur par son poids, et additionner.

Exemple : convertir $(90F)_{16}$ en décimal :

Poids	4'096	256	16	1
Symbole		9	0	F
Valeur		9	0	15



$$9 \times 256 + 0 \times 16 + 15 \times 1 = 2'319$$

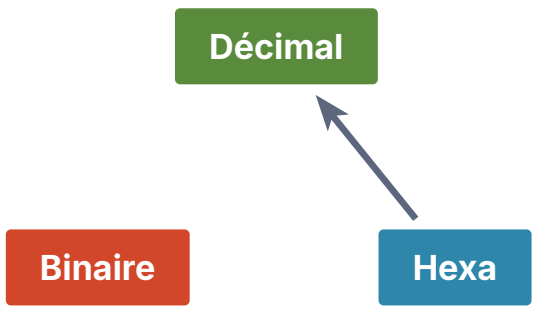
$$(90F)_{16} = (2'319)_{10}$$

Hexadécimal → décimal : un 2e exemple

Hexa	Décimal
0	0
1	1
2	2
3	3
4	4
5	5
6	6
7	7
8	8
9	9
A	10
B	11
C	12
D	13
E	14
F	15

Convertir $(B33F)_{16}$ en décimal :

Poids	4'096	256	16	1
Symbole	B	3	3	F
Valeur	11	3	3	15

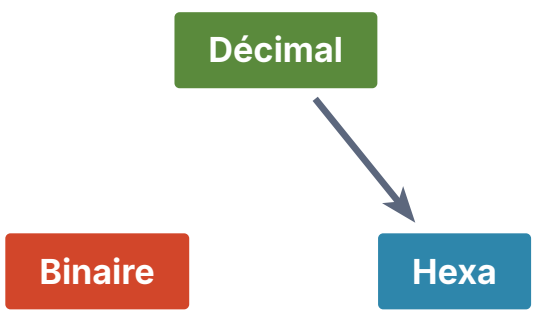


$$11 \times 4'096 + 3 \times 256 + 3 \times 16 + 15 \times 1 = 45'887$$

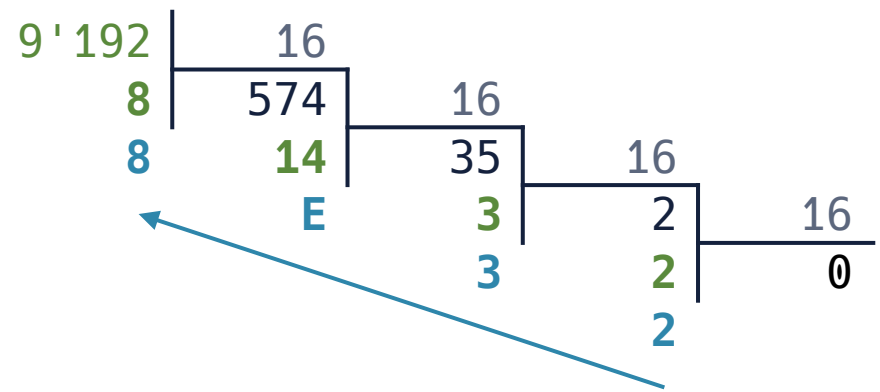
$$(B33F)_{16} = (45'887)_{10}$$

Décimal → hexadécimal, directement

1. Diviser par 16, puis diviser le quotient par 16, et ainsi de suite, jusqu'à ce que le **quotient** soit 0.
2. Convertir chaque **reste**, de 0 à 15, en son **symbole** hexadécimal.
3. Collecter les symboles du bas vers le haut.



Exemple : convertir $(9'192)_{10}$ en hexadécimal :



Résultat : $(9'192)_{10} = (23E8)_{16}$

Exercices : hexadécimal

Binaire	Hexa	Décimal
0000	0	0
0001	1	1
0010	2	2
0011	3	3
0100	4	4
0101	5	5
0110	6	6
0111	7	7
1000	8	8
1001	9	9
1010	A	10
1011	B	11
1100	C	12
1101	D	13
1110	E	14
1111	F	15

Activité · 10 min

Avec les deux tableaux :

1. Écrire 70 en hexadécimal.
2. Convertir $(B7)_{16}$ en décimal :
   a) d'abord en passant par le binaire ;
   b) puis directement, avec la grille des poids.
3. Combien de symboles hexadécimaux faut-il pour écrire 8 bits, c'est-à-dire les nombres de 0 à 255 ?
   Par exemple, $(E)_{16}$ utilise 1 symbole, $(ACAD)_{16}$ en utilise 4.

Poids	4'096	256	16	1
Symbole				
Valeur				

Exercices : hexadécimal

1. Écrire 70 en hexadécimal.

8 bits, soit un octet

128	64	32	16	8	4	2	1
0	1	0	0	0	1	1	0

Décimal	Calculs
70	64 + 4 + 2

0100 0110
4 6

Résultat : $(70)_{10} = (46)_{16}$

Exercices : hexadécimal

2. Convertir $(B7)_{16}$ en décimal.

a) En passant par le binaire : chaque symbole donne 4 bits.

B	7
1011	0111

8 bits, soit un octet

128	64	32	16	8	4	2	1
1	0	1	1	0	1	1	1

Décimal	Calculs
183	128 + 32 + 16 + 4 + 2 + 1

b) Directement, avec la grille des poids :

Poids	4'096	256	16	1
Symbole			B	7
Valeur			11	7

$$11 \times 16 + 7 \times 1 = 183$$

$$(B7)_{16} = (183)_{10}$$

Exercices : hexadécimal

3. Combien de symboles hexadécimaux faut-il pour écrire 8 bits, c'est-à-dire les nombres de 0 à 255 ?

Par la définition

- Il faut **8 bits** pour écrire les nombres de 0 à 255.
- Un symbole hexadécimal représente **4 bits**.
- Deux symboles représentent donc 8 bits.

Par le calcul

Le plus petit et le plus grand nombre avec deux symboles hexa :

Poids	16	1
Symbole	0	0
Valeur	0	0

$0 \times 16 + 0 \times 1 = 0$

Poids	16	1
Symbole	F	F
Valeur	15	15

$15 \times 16 + 15 \times 1 = 255$

Réponse : **2 symboles**, de $(00)_{16} = (0)_{10}$ à $(FF)_{16} = (255)_{10}$.

Les nombres négatifs en hexadécimal

On passe par le binaire, en complément à deux, puis on groupe les bits par 4.

En complément à deux · sur 8 bits

```
          −4
1111  1100
  F     C
```

$(-4)_{10} = (FC)_{16}$

Sans signe · sur 8 bits

```
         252
1111  1100
  F     C
```

$(252)_{10} = (FC)_{16}$

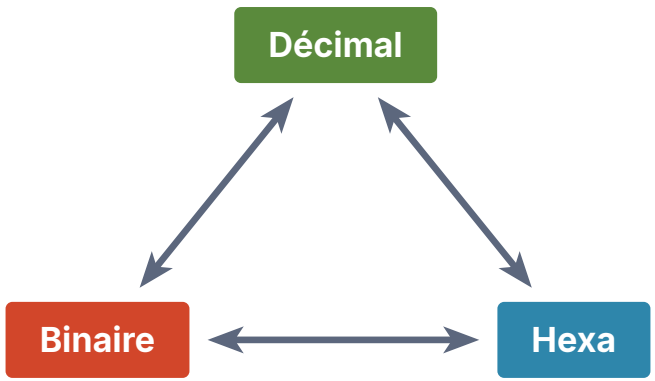
Les mêmes bits, deux interprétations : $(FC)_{16}$ vaut -4 ou 252.
En complément à deux, sur 8 bits, un nombre négatif commence par un symbole de 8 à F.

Récapitulatif : convertir entre les trois bases

Binaire ↔ décimal

→ **décimal** : additionner les poids des bits à 1

→ **binaire** : soustractions successives, ou divisions successives par 2



Binaire ↔ hexadécimal

Par groupes de 4 bits, en utilisant le tableau des 16 symboles.

Hexadécimal ↔ décimal

→ **décimal** : chaque symbole en valeur, multipliée par son poids (une puissance de 16), puis additionner

→ **hexadécimal** : divisions successives par 16, chaque reste en symbole

Un convertisseur entre les trois bases : ri.nwolff.info/binary-converter

Sources

Contenu et outils

Convertisseur entre binaire, décimal et hexadécimal
ri.nwolff.info

The Almighty Guru – le nombre 255 dans les jeux vidéo
thealmightyguru.com

Images

Konstantin Lanzet – une mémoire à tores magnétiques (CC BY-SA 3.0)
commons.wikimedia.org

Geoffrey Saxe – le comptage des Oksapmin
culturecognition.com

Joeykentin – l'os d'Ishango (CC BY-SA 4.0)
commons.wikimedia.org

Hellbus – un compteur kilométrique (domaine public)
commons.wikimedia.org

M. Morris Mano et Michael D. Ciletti – *Digital Design*, l'additionneur-soustracteur de 4 bits

Roland Balik, U.S. Air Force – une partie de « Qui est-ce ? » (domaine public)
commons.wikimedia.org